

The Referral Bridge: Digital Workflow Optimization and Offline Synchronization System

Hospital Site: Holy Family Catholic Hospital

Prepared By: Azumah Ernest, Software Developer

Date: September 10, 2026

1. Problem Identification

During operational field observations of the administrative workflows at Holy Family Catholic Hospital, a severe data bottleneck was identified in how patient referral data moves between the billing, clinical, and information technology departments. The existing workflow relied entirely on manual, paper-based steps. This process slowed down file handoffs, introduced significant administrative stress, and created a high risk of data entry errors given the volume of referrals handled daily.

2. Problem Statement

The hospital processes over 2,000 patient referrals every month. The legacy workflow required the claims unit to extract patient details from the claims system into an Excel sheet, which was printed and manually transcribed onto small physical referral forms. These paper forms were carried to two separate medical officers to collect physical ink signatures for the "Referral From" and "Referral To" fields.

After obtaining both signatures, the physical forms were sent to the IT department. IT personnel used mobile phone cameras to photograph each form, manually renamed each image to match the patient, searched for the patient's folder inside the third-party EHR web portal (LHIMS), and uploaded the file. This loop was slow, disorganized, and offered no real-time way for the team to track which files were completed, leading to duplicated effort and missing records.

3. Recommended Solution

To eliminate these vulnerabilities, I designed and developed The Referral Bridge, a centralized web application built with React and Firebase (Authentication and Firestore). The solution digitizes the data pipeline from intake to upload without requiring dedicated local server hardware.

Digital Data Intake

The IT department uploads the claims Excel export directly into the React app, which parses it client-side and batch-creates the corresponding referral records in Firestore. Writes are chunked into batches of 450 documents at a time, staying safely under Firestore's 500-write-per-batch limit, so a full month's intake commits reliably in a handful of requests instead of one write per referral.

Dynamic Referral Form Generation

Doctors review the pre-populated referral form on their dashboard. Approving it renders a print-ready form using the HTML5 Canvas API, sized to match the hospital's physical 150mm × 100mm landscape form, populated with the patient's details and the two doctors' saved signature images. Each signature is cropped to its inked strokes using alpha-transparency detection, recolored to a consistent ink-blue, and

drawn with a small tilt and offset derived deterministically from the signature's own file reference, so the same signature always renders the same way on every download, while avoiding the flat, "rubber-stamped" look of a perfectly centered, perfectly level image pasted onto the form. The finished canvas is exported directly as a JPEG file, matching the format LHIMS expects for upload.

Admin Task Allocation

An administrative dashboard allows supervisors to select signed referrals and allocate them to specific IT officers. Each referral carries an assignment field, so an officer's dashboard only ever shows the batch of forms allocated to them, keeping task distribution clean and avoiding duplicated work across the team.

4. Implementation and Technical Challenges Solved

Challenge A: Eliminating Physical Paper and Tracking Real-Time Progress

Going entirely paperless removed the physical stack of forms the IT team previously used to see what was pending or completed. Without that physical cue, a digital method was needed to coordinate progress and prevent multiple officers from uploading the same patient file.

Solution: I implemented a dedicated Officer role inside the React application. When an officer logs in, they see only the batch of forms assigned to them. In practice, the officer runs the React app alongside the LHIMS portal: clicking "Copy ID" copies the patient number to paste into LHIMS, "Download Form" saves the generated JPEG, and once the file is uploaded to LHIMS, "Mark Done" removes the referral from Firestore (and increments a running completed-referrals counter used for reporting). The record disappears from the officer's screen immediately, giving the team a clean, real-time view of remaining work with zero duplicated effort.

Challenge B: Offline Local Operations over the Hospital LAN

The LHIMS web application operates on the hospital's local area network, and workstations frequently run without active external internet access. The React application needed to let IT officers copy patient IDs, download generated forms, and mark referrals done completely offline, without losing data or breaking the connection to Firebase once it returned.

Solution: This required two distinct fixes once the actual offline behaviour was tested end to end.

- **Durable offline data and sync.** Firestore's default web client keeps its cache in memory only, which works within a single browser session but is wiped by a page reload, a routine event on a LAN prone to drops. I switched the app to Firestore's built-in persistent local cache, backed by IndexedDB, with multi-tab support enabled. Assigned referrals now stay available after a reload while offline, and a "Mark Done" action taken offline queues locally and is replayed to Firestore automatically the moment the connection returns. This relies on the offline engine already built into the Firestore SDK rather than a custom caching layer.
- **Reliable form generation offline.** The generated referral JPEG includes the hospital crest, which was originally loaded as a separate image file fetched over the network at the moment the form was generated. If that specific file was not already sitting in the browser's cache, the fetch silently failed while offline and the crest was missing from the downloaded form, even though the rest of the form rendered correctly. This was fixed by embedding the crest as a base64 image directly inside the

application bundle at build time, so generating the form no longer depends on a network request at all, online or offline.

“Copy ID” required no change, since it only uses the browser's clipboard API and was never dependent on network access.

5. Conclusion

This is how I turned days of hard work into a few clicks. By replacing a paper trail that ran through three departments with a system that does the same work in real time, the hospital saves the cost of printing over 2,000 referral forms every month, and staff reclaim time that used to go into manual transcription, photographing, and filing, time they can now spend on other important duties.

Sometimes it takes just curiosity and the right mindset to arrive at solutions like this one. Often the hardest part is not that the solution is difficult to build, but that the problem is so familiar nobody stops to ask whether it could be solved at all.